Paper 8 — Why Networks Learn One Unit at a Time

Yedi Zhang, Andrew Saxe, Peter E. Latham (ICLR 2026)

Saddle-to-Saddle Dynamics Explains A Simplicity Bias Across Neural Network Architectures

Key: zhang2026saddle | Tags: G, D, A / R, B | Tier: Must read

The paper in one minute

Question: Why do networks often learn simple functions first, pausing at plateaus before suddenly fitting more complex patterns?
Model/Method: Analyze fixed points, invariant manifolds and gradient-flow dynamics across fully connected, convolutional and attention-based models.
Finding: Training can move between nested saddles, increasing the number of *effective* neurons, kernels or attention heads. Linear models develop low-rank structure from data-dependent growth rates; quadratic-weight models develop sparse units from initialization-dependent growth rates.
Why we care: Plateaus can mark successive stages of representation learning rather than merely failed optimization.

1. First understand the strange observation: a large network can behave like a small one

Suppose a trained width-10 network uses only one effective neuron. It may be capable of fitting a more complex function, but its current weights implement a function expressible with a width-1 network. The authors call this *simplicity*: the smallest number of active/effective units required to express the current input–output map, *not* low test error or a small parameter count.



The sequence can continue: each stage often adds one or a few effective units. The schematic illustrates the mechanism, not a universal training trajectory. See the paper's **Figure 1**, pp. 2–3.

2. A worked example: how a width-1 solution sits inside width-2

For a scalar-output two-layer model, write

$$f(x) = v_1\phi(x;u_1) + v_2\phi(x;u_2).$$

Start with a stationary *width-1* solution $f_*(x) = v\phi(x;u)$. Create a wider representation by duplicating its unit:

$$u_1 = u_2 = u, \qquad v_1 = v_2 = \frac{v}{2}.$$

Then $f(x) = \frac{v}{2}\phi(x;u) + \frac{v}{2}\phi(x;u) = f_*(x)$: the network is wider, but the represented function is unchanged. Because the old unit had zero derivatives and the new units share or scale those derivatives, the duplicated configuration is also stationary under the paper's setup.

Why this example matters

A stationary point of a narrower model can reappear in a larger model. That is the **embedded fixed-point** principle (Theorem 1). But an embedded fixed point is *not automatically a saddle* in every architecture; it may instead be a local minimum.

3. Geometry: embedded fixed points versus invariant manifolds

Theorem 1 (embedded fixed points). For the network class in Eq. (1), any fixed point of a width-$(H-1)$ model can be represented as a fixed point of a width-H model with the same input–output map. Corollary 2 extends this to additional units and layers. Different constructions require different properties of the activation (zero outputs, homogeneity or linearity).

Theorem 3 (invariant manifolds). Certain relations between units are preserved by *gradient flow*. For example, if two units start with identical parameter vectors $\theta_1 = \theta_2$, then their gradients are identical, so

$$\frac{d}{dt}(\theta_1 - \theta_2) = -\nabla_{\theta_1} L + \nabla_{\theta_2} L = 0.$$

Thus identical units remain identical at all later times. A wider network restricted to that manifold behaves like a narrower network. For certain activations, zero, proportional or linearly dependent units also form such manifolds.

Difference to memorize

A **fixed point** is a single stationary configuration ($\dot{\theta} = 0$). An **invariant manifold** is a whole set of configurations that gradient flow cannot leave once it enters. The paper's story uses both: saddles define the stages; approximately invariant low-width paths help explain the transitions.

4. Where the mathematics becomes dynamical: linear two-layer networks

The network's effective linear map is

$$f(x) = \sum_{i=1}^{H} v_i u_i^\top z(x) = W z(x), \qquad W = \sum_i v_i u_i^\top.$$

Under squared-error gradient flow (paper's Eq. 9),

$$\dot{v}_i = (\Sigma_{yz} - W\Sigma_{zz})u_i, \qquad \dot{u}_i = (\Sigma_{yz} - W\Sigma_{zz})^\top v_i.$$

When every weight is initially tiny, $W = O(\varepsilon^2)$; the residual terms involving W can initially be neglected. The resulting linearized dynamics grow along singular directions of Σ_{yz} at rates determined by its singular values s_k.

5. Worked numbers: why data modes learn at different speeds

In an illustrative two-mode case, let $s_1 = 3$, $s_2 = 1$ and suppose each unstable growing-mode amplitude begins at 0.01. The early linearized dynamics give

$$a_1(t) \approx 0.01 e^{3t}, \qquad a_2(t) \approx 0.01 e^{t}.$$

The first mode reaches unit size when $t = \log(100)/3 \approx 1.535$. At that same time,

$$a_2(t) \approx 0.01 e^{1.535} \approx 0.0464.$$

So one direction has become large while the other remains small. The network's weights are *approximately low-rank* and it behaves like an effectively narrower model.

What Theorem 4 proves beyond this toy calculation

If the top singular value has multiplicity $r < D$ and the initialization is independent small Gaussian noise, then when the top-r component becomes $O(1)$, the other component is $O(\varepsilon^{1-s_{r+1}/s_1})$ almost surely for the *linearized* system. Distinct singular values produce a separation of learning times. This is not a theorem for every nonlinear network.

6. The second mechanism: initialization makes one unit win first

The linear case separates *data directions*: a large singular value learns first. In the paper's quadratic-weight case (including its linear self-attention construction), the early growth can separate *individual units*. Proposition 5 formalizes this under specified symmetry/eigenvalue and small-Gaussian-initialization conditions.

To see the intuition, the authors consider the simplified scalar differential equation $\dot{v}_i = v_i^2$. Separate variables:

$$\frac{dv_i}{v_i^2} = dt \quad \Longrightarrow \quad v_i(t) = \frac{1}{1/v_i(0) - t}.$$

Imagine $v_1(0) = 0.10$ and $v_2(0) = 0.05$. By $t = 9$,

$$v_1(9) = \frac{1}{10 - 9} = 1, \qquad v_2(9) = \frac{1}{20 - 9} \approx 0.091.$$

The slightly larger unit races ahead while the other stays relatively small. Unlike the exponential mode story, this is a *rich-get-richer* mechanism driven by initial differences between units. The scalar example is pedagogical; the paper's exact quadratic-network equations and Proposition 5 are more involved.

Do not mix the two mechanisms

Linear-weight case: different data singular values → different growth rates across directions → low-rank structure.
Quadratic-weight case: different initial unit magnitudes → different growth rates across units → sparse use of units.

7. What Figure 1 and Figure 2 actually demonstrate

Figure / test	Correct interpretation
Fig. 1A	Cartoon: near an embedded saddle the loss plateaus; a change of effective-width manifold allows another stage.
Fig. 1B–G	Examples across linear fully connected/convolutional, ReLU fully connected/convolutional, linear self-attention and quadratic networks. The illustrated transition goes from one to two effective units.
Fig. 2A	Increasing width barely changes the stage timing in the tested linear case, but can shorten plateaus for the tested quadratic / attention case.
Fig. 2B	Closing the data singular-value gaps shortens or eliminates intermediate plateaus in linear networks; in quadratic models the initialization mechanism can preserve them.
Fig. 2C–D	Starting near a low-width manifold, and the scale of initialization, influence whether/how prominently stages appear. Larger random initialization weakens plateaus.

8. Main contributions worth remembering

1. **A cross-architecture geometric framework:** nested fixed points of narrower subnetworks and invariant manifolds of lower effective width.
2. **A mechanism for simplicity bias:** stage-like learning arises when trajectories remain near simple-function manifolds and intermittently recruit extra effective units.
3. **Two explanations of timescale separation:** data-induced rank bias versus initialization-induced sparsity.
4. **Testable predictions:** data spectrum, architecture width and initialization control plateau duration and occurrence, with supporting numerical experiments.

9. Important assumptions and limitations

Aspect	What you must qualify in the survey
Training rule	Formal invariance is analyzed for continuous-time **gradient flow**, a small-step limit of GD, not general minibatch SGD.
Architecture	The general Eq. (1) has additive unit structure; different fixed-point constructions require specific activation properties.
Dynamics	Detailed growth-rate results in Section 5 focus on **two-layer** linear- or quadratic-in-weights models with squared-error loss and small random initialization.
Saddle geometry	Embedded *fixed points* exist generally; their saddle character and attraction of trajectories need further model-dependent conditions.
Generality	A full dynamical theory for arbitrary deep models/transformers is **not** proved; some extensions are explicitly conjectural.
Not inevitable	The paper discusses tanh and large-initialization settings in which saddle-to-saddle stages may be absent.

One subtlety for a good survey

Theorem 1/Corollary 2 and Theorem 3 are general *structural existence/invariance* claims for the stated architecture class. The claim that training *typically follows* these low-dimensional manifolds is developed using specific dynamical mechanisms and experiments. Never present the latter as a universal convergence theorem.

10. Connections to the papers we have already read

Earlier paper	How Zhang's perspective differs
Saxe (2014)	Stage-like mode learning in deep linear nets; Zhang unifies it with effective-unit recruitment.
Dauphin (2014)	Saddle plateaus as obstacles; Zhang adds their role in progressively learned representations.
Lee (2016)	Avoiding *convergence* to saddles differs from repeatedly passing *near* saddles.
Jin (2017)	Perturbations provide escape bounds; Zhang studies natural structured transitions without intervention.
Li (2018)	Architecture shapes the visible loss landscape; Zhang explains architecture-dependent trajectories.
Du (2019)	Width can enable successful optimization; Zhang studies which units become effectively active.
Galli (2026)	Large steps and flatness change dynamics; Zhang instead studies progressive complexity.

11. Quick reference and reading route

Term / item	Remember this
Fixed point	$\dot{\theta} = -\nabla L(\theta) = 0$; stationary, not necessarily globally optimal.
Invariant manifold	A set preserved by gradient flow (e.g., equal units remain equal).
Effective width	Minimum number of units needed to express the current learned function.
Theorems 1 and 3	Narrow fixed points embed into wider networks; some simple-weight relations remain invariant.
Theorem 4 / Prop. 5	Different data directions grow at different rates / different units grow at different rates.
Read first	Abstract and Figure 1; Sections §3–§6; discussion §7. Use Appendices E–H only to verify proof steps.

Papers Master — 2-line note

Zhang et al. (2026) explain progressive complexity learning through embedded stationary points and approximately low-width invariant manifolds. Linear models show data-induced rank growth; quadratic-weight models show initialization-induced unit sparsity. Detailed trajectory guarantees require restricted settings.

Original paper: ICLR 2026 proceedings PDF. Worked examples are teaching adaptations, not quoted numerical experiments.